

EmpowerHR — System Architecture & Data Flow Guide

Project Overview & Tech Stack

```
Frontend (React + Vite)
  |
  ▼ HTTP JSON / Requests
Backend API (Express.js on Node.js)
  |
  ▼ Prisma ORM
Database (PostgreSQL)
```

Technology Breakdown

- **Frontend:** React, Vite, React Router, Vanilla CSS.
- **Backend:** Node.js, Express.js, Prisma ORM.
- **Database:** PostgreSQL (runs locally via Docker on port 5433).
- **Authentication:**
 - **Local Dev:** Mock login via x-employee-id header (switch between seeded personas in 1 click).
 - **Production:** Microsoft Entra ID (Azure AD) via RS256 JWT tokens.

User Roles & Permissions (RBAC)

There are 3 user roles in the database. Roles are **never sent from the frontend**; the backend looks up the logged-in user in PostgreSQL on every request to verify their role:

1. **EMPLOYEE** : apply for leaves, view leave balances, cancel pending leaves.
2. **MANAGER** : Can do everything an Employee does, plus: approve/reject/request info on leave requests from their **direct reports**, see their team leave calendar, and create goals/reviews for their direct reports.

3. **HR** : Can do final level-2 approvals on all leave requests, configure leave types and carry-forward rules, manage org settings, view company-wide analytics, and trigger year-end balance resets.

Database Schema & Models (`schema.prisma`)

Here is how data is stored in PostgreSQL and connected together:

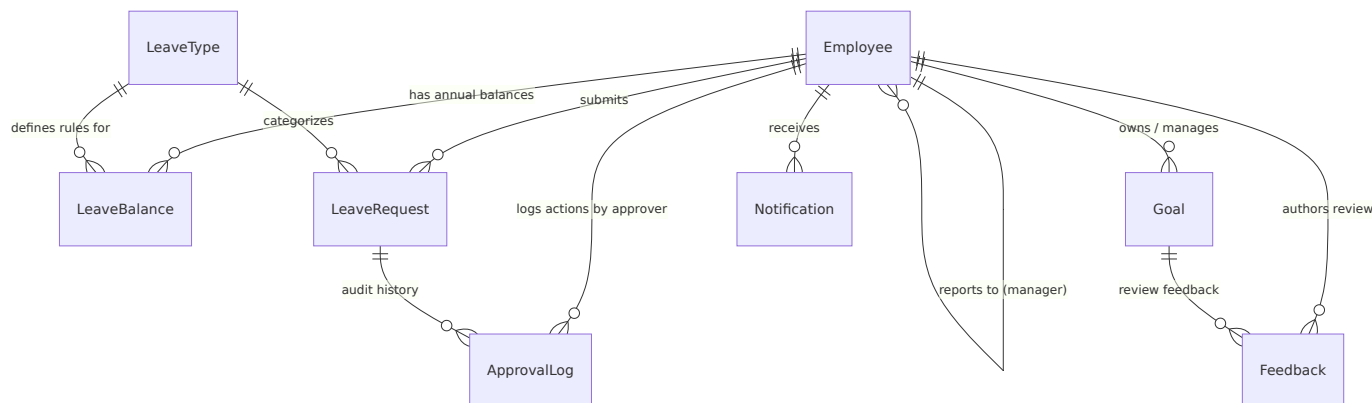


Table Summaries:

- **Employee** : User profile with `id` , `employeeId` (`EMP0001`), `name` , `email` , `department` , `role` (`EMPLOYEE` / `MANAGER` / `HR`), and `reportingManagerId` (links to their manager's `id`).
 - **LeaveType** : Types of leave (Casual, Sick, Earned, Unpaid) with settings like `defaultAnnualEntitlement` , `isPaid` , `unlimited` , `requiresAttachment` , and `carryForwardMaxDays` .
 - **LeaveBalance** : Tracks days for each employee per leave type per year:
 - $\text{Remaining Balance} = \text{allocated} + \text{carriedOver} - \text{used}$
 - **LeaveRequest** : The leave application. Stores `startDate` , `endDate` , `days` (working days), `reason` , `attachmentUrl` , `status` , and `currentApproverStage` .
 - **ApprovalLog** : Immutable history trail. Every time a request is submitted, approved, rejected, info-requested, or cancelled, a new row is inserted here.
 - **Goal** & **Feedback** : Performance goals assigned by managers to employees, with reviews (rating 1–5 stars and comments).
 - **OrgSettings** : System-wide settings (e.g. `currentCycleYear`: `2026` , `allowSelfAssessment`: `false`).
-

Authentication Flow: How the Backend Knows Who You Are

Every API request passes through the `attachUser` middleware in `src/middleware/auth.js`:

Incoming Request

```
|
|  ↳ 1. Check Header (x-employee-id: "EMP0002" OR Bearer JWT Token)
|  ↳ 2. Look up Employee in PostgreSQL
|  ↳ 3. Query Direct Reports: WHERE reportingManagerId = user.id
|  ↳ 4. Attach to Request:
|      req.user = { id, name, role, department, ... }
|      req.directReportIds = ["id_1", "id_2"]
|
|  ▼ Next Middleware / Route Handler
```

- If you are logged in as a Manager, `req.directReportIds` gives the API a ready list of your team members.
- If a route needs you to be logged in, `requireAuth` checks if `req.user` exists.
- If a route needs HR rights, `requireRole('HR')` checks `req.user.role === 'HR'`.

Step-by-Step Feature Data Flows

Flow 1: Employee Applies for Leave

```

[Frontend: ApplyLeaveModal]
|
▼ POST /api/leave-requests
[Backend: leaveService.createLeaveRequest]
|
|→ 1. Validate dates (endDate >= startDate)
|→ 2. Count Business Days (skips Sat & Sun)
|→ 3. Check attachment (if leave type requires doctor note /
document)
|→ 4. Check for overlapping dates with existing active leaves
|→ 5. Check remaining balance (allocated + carriedOver - used >=
requested)
|→ 6. Determine stage:
|    - Has manager? → status: PENDING_MANAGER, stage: MANAGER
|    - No manager? → status: PENDING_HR, stage: HR
|
▼ Database Transaction (prisma.$transaction)
|→ INSERT INTO LeaveRequest
|→ INSERT INTO ApprovalLog (action: SUBMIT)
|
▼ Send in-app notification to Manager
|
▼ Response: 201 Created (JSON with leave request & logs)

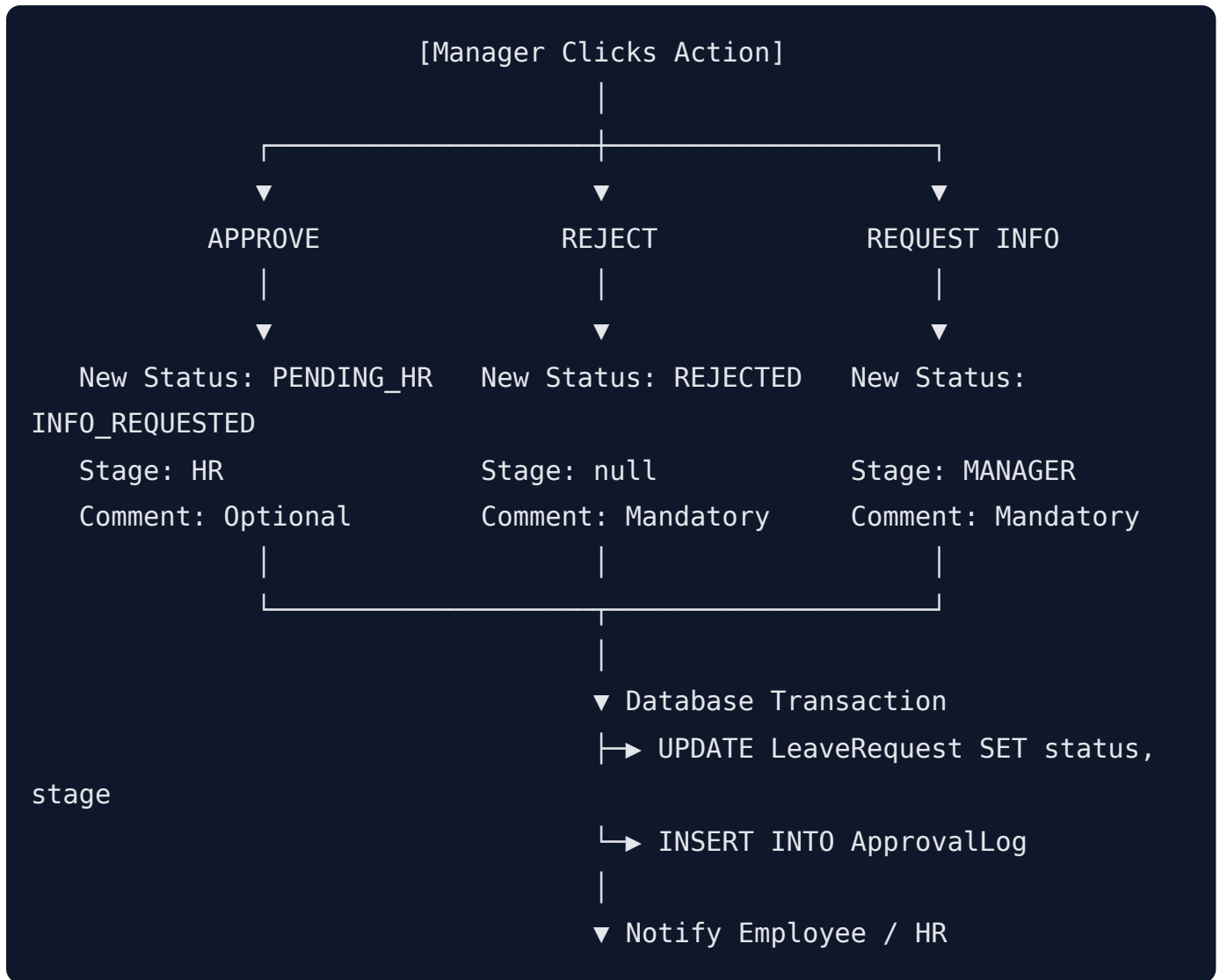
```

What happens if validation fails?

- If the dates are on a weekend: returns 400 "The selected range contains no working days".
- If an active leave already covers those dates: returns 409 "You already have a leave request overlapping these dates".
- If requested days exceed available quota: returns 409 "Insufficient balance".

Flow 2: Manager Reviews Leave (Level 1)

When a manager opens their **Pending Approvals** tab, they see requests where `status = 'PENDING_MANAGER'`.



- **If Approved:** The request moves to `PENDING_HR`. It now shows up in HR's approval queue.
- **If Info Requested:** The employee gets a notification and can reply via `POST /api/leave-requests/:id/respond-info`. Once they reply, the status returns to `PENDING_MANAGER`.

Flow 3: HR Final Approval & Balance Deduction (Level 2)

HR admins review requests where `status = 'PENDING_HR'`.

[HR Admin Clicks Approve]

|
▼ POST /api/approvals/:id/approve

[Backend: leaveService.transitionLeaveRequest]

|
|→ 1. Check actor is HR (actor.role === 'HR')
|→ 2. State machine verifies transition is valid
|

▼ Database Transaction (prisma.\$transaction)

|→ A. Update LeaveBalance:

| UPDATE LeaveBalance
| SET used = used + request.days
| WHERE employeeId = req.employeeId AND cycleYear = 2026

|→ B. Update LeaveRequest:

| UPDATE LeaveRequest
| SET status = 'APPROVED', stage = null, decidedAt =

NOW()

|→ C. Insert ApprovalLog (action: APPROVE, stage: HR)

|
▼ Notify Employee: "Your leave request has been approved!"

Why is balance deducted only at HR approval?

If balance was deducted when applying, and the request got rejected or cancelled, we would have to add logic to refund it. Deducting atomically on final approval keeps balance math clean and avoids race conditions.

Flow 4: Employee Cancels a Leave

An employee can withdraw any request before it reaches a finished state:

[Employee Clicks Cancel] → POST /api/leave-requests/:id/cancel

|

▼

Update LeaveRequest: status = 'CANCELLED'

Insert ApprovalLog: action = 'CANCEL'

Flow 5: Performance Goals & Feedback

1. Manager creates goal:

```
POST /api/goals { employeeId: "emp_2", title: "Ship Feature X",  
reviewCycle: "2026-H1" }
```

→ Saved to DB (status: NOT_STARTED), Employee notified.

2. Employee works on goal:

```
PATCH /api/goals/:id { status: "IN_PROGRESS" }
```

3. Manager submits review:

```
POST /api/goals/:id/feedback { rating: 5, comments: "Great job on  
delivery!", authorType: "MANAGER" }
```

→ Feedback record inserted with 1-5 star rating.

4. Employee Self-Assessment (Optional):

```
POST /api/goals/:id/feedback { comments: "Completed all milestones",  
authorType: "SELF" }
```

→ Checked against OrgSettings.allowSelfAssessment. If false, returns 403 Forbidden.

Flow 6: Year-End Cycle Reset (Carry-Forward & New Quota)

At the end of the year, HR triggers the rollover job (`POST /api/settings/cycle-reset`):

For each active Employee and each LeaveType:

1. Calculate Remaining = allocated + carriedOver - used

2. Calculate CarriedForward = min(Remaining,

leaveType.carryForwardMaxDays)

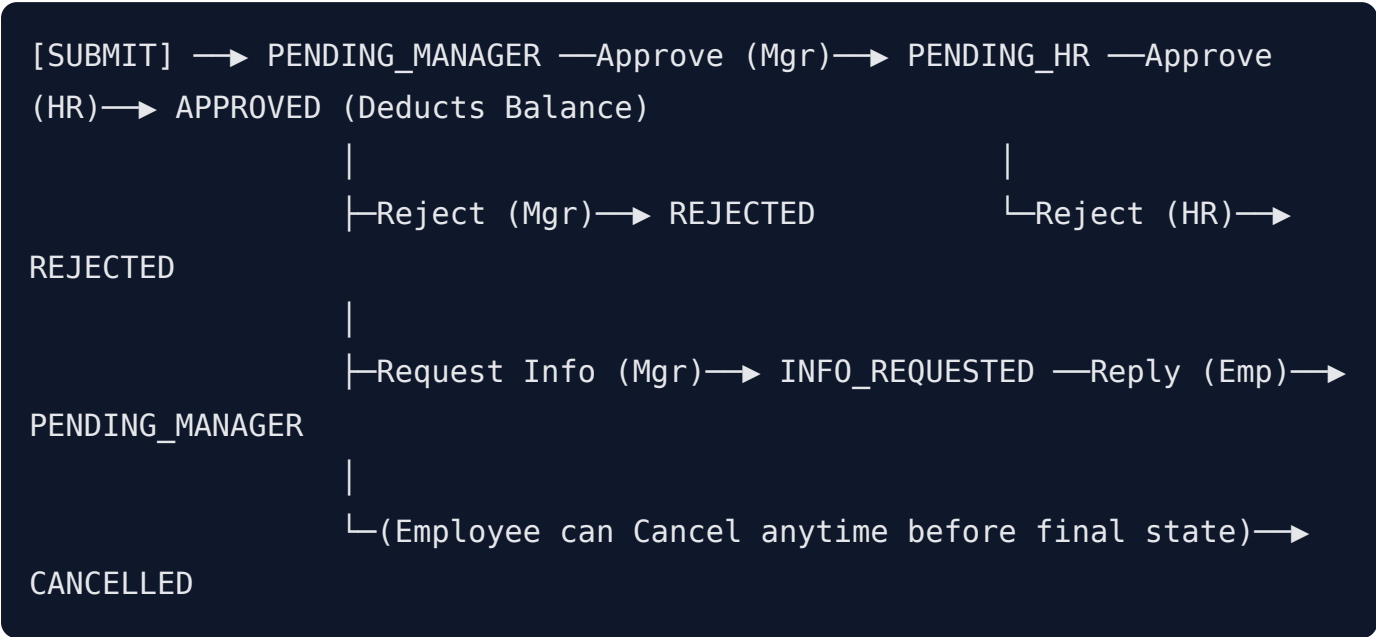
3. Calculate New Year Entitlement (pro-rated if the employee joined mid-year)

4. Bulk upsert new LeaveBalance rows for (Year + 1)

5. Advance OrgSettings.currentCycleYear to (Year + 1)

Leave Request State Machine

The status of a leave request moves in one direction through clear states:



Complete API Route Map

Method	Endpoint	Allowed Roles	Description
GET	/api/auth/users	Public	List test users (Mock dev mode)
GET	/api/me	Authenticated	Get current logged-in user profile & direct report IDs
GET	/api/leave-requests	Authenticated	List leave requests (scoped: Employee=Self, Manager=Team, HR=All)
POST	/api/leave-requests	Authenticated	Submit a new leave request (supports file attachment upload)
GET	/api/leave-requests/:id	Authenticated	View details of a leave request and its audit logs

POST	/api/leave-requests/:id/cancel	Request Owner	Cancel a pending leave request
POST	/api/leave-requests/:id/respond-info	Request Owner	Reply to a manager's request for more information
GET	/api/approvals/pending	Manager, HR	Get pending requests waiting for your approval stage
POST	/api/approvals/:id/approve	Manager, HR	Approve request (Level 1 as Manager, Level 2 as HR)
POST	/api/approvals/:id/reject	Manager, HR	Reject request (requires comment)
POST	/api/approvals/:id/request-info	Manager	Request more details from employee (requires comment)
GET	/api/goals	Authenticated	List goals (Employee=Self, Manager=Team, HR=All)
POST	/api/goals	Manager	Create a goal for a direct report
PATCH	/api/goals/:id	Manager, Owner	Update goal status or description
POST	/api/goals/:id/feedback	Manager, Owner	Submit rating & review or self-assessment
GET	/api/goals/completion-stats	HR	View review completion percentage per manager
GET	/api/analytics/overview	Manager, HR	Overview stats (approval rates, total days, response times)

GET	/api/leave-types	Authenticated	List active leave types and their rules
POST	/api/leave-types	HR	Create / update leave types and carry-forward caps
GET	/api/settings	Authenticated	Get current org settings & cycle year
PUT	/api/settings	HR	Update org settings (e.g. toggle self-assessment)
POST	/api/settings/cycle-reset	HR	Run end-of-year carry-forward & balance reset job

End of Guide. Ready to export to PDF.